DIAGNÓSTICO DE APRENDIZAJE: MISIÓN NEXUS (CÓDIGO A CONOCIMIENTO)

1. Fortalezas demostradas

- ***Rutas y gestión de archivos en GitHub:*** *Comprendiste el uso del carácter / en la interfaz web para crear subcarpetas al vuelo y estructurar la jerarquía del repositorio.*
- ***Integración de referencias externas:*** *Comprendiste la diferencia entre almacenar binarios pesados e integrar documentación mediante enlaces externos (DeepWiki) o almacenamiento de código en texto plano.*
- ***Sintaxis de enlace en Markdown:*** *Aplicaste correctamente el uso de rutas relativas (/ y ..) e hipervínculos [texto](url) para conectar documentos dentro del repositorio.*
- ***Diferenciación de flujos de trabajo:*** *Identificaste la diferencia operativa entre modificar archivos directamente en la interfaz web de GitHub.com y realizar cambios locales con sincronización en GitHub Desktop.*

2. Falencias observadas

- ***Distinción entre subir un archivo y almacenar su contenido:*** *Presentaste confusión recurrente sobre la diferencia entre subir un archivo como objeto en el repositorio (ej. un .pdf o .drawio) y copiar/pegar el texto plano o código XML dentro de un documento existente (ej. un .md o .txt).*
- ***Visualización de diagramas .drawio:*** *Dificultad para discernir cómo GitHub procesa el código XML nativo de un archivo .drawio en comparación con un archivo de imagen exportado (.png / .svg).*

3. Diagnóstico del Hito 3

Concepto	***Clasificación***	***Justificación basada en la conversación***
POO (Concepto General)	***Comprensión demostrada***	*Se justificó correctamente la separación de responsabilidades entre clases (Cadete para datos/energía vs. Inventario para objetos) frente a un método monolítico.*
Clases y Objetos	***Comprensión demostrada***	*Se definió la clase como un molde y el objeto como una instancia, calculando correctamente la mutación del estado (Energia:* $60 - 10 = 50$*).*

Herencia y Polimorfismo	***Comprensión demostrada***	*Se explicó la ejecución dinámica del método Actuar() sobre Robot o Enemigo invocados mediante la referencia abstracta Personaje.*
Interfaces	***Comprensión demostrada***	*Se comprendió la desacreditación de herencia rígida mediante la interfaz IInteractuable, permitiendo que clases disímiles (Terminal, Puerta) implementen Interactuar().*
Listas Genéricas y LINQ	***Comprensión demostrada***	*Se validó la filtración condicional (Where(o => o.Poder >= 20)) y proyección (Select(o => o.Nombre)), descartando elementos fuera de rango.*
Constructores	***Evidencia insuficiente***	*No se evaluó ni debatió explícitamente la sobrecarga o inicialización en el cuerpo de los constructores en las interacciones directas.*
Encapsulamiento	***Comprensión parcial***	*Se reconoció el acceso a propiedades como Energia, pero no se profundizó en modificadores de acceso (private, protected) dentro del código directo aportado.*

4. Mi forma de aprender

- ***Enfoque práctico e interactivo:*** *Consultas guiadas por necesidades operativas inmediatas ("¿cómo creo una carpeta?", "¿cómo agrego el enlace?").*
- ***Iteración por aclaración:*** *Cuando una explicación general no satisfizo tu necesidad exacta (ej. diferenciar subir un archivo vs. escribir dentro de uno), reformulaste la pregunta hasta obtener el procedimiento exacto.*
- ***Validación estructurada:*** *Organizaste tus notas en formato de preguntas y respuestas estructuradas para preparar tu evaluación técnica.*

5. Plan de estudio (5 Conocimientos prioritarios)

1. Formatos de almacenamiento vs. Código en Git (Prioridad Alta)

- ***Ejercicio práctico:*** *Crea una subcarpeta llamada diagramas/ en GitHub. Sube un archivo .drawio original y, en la misma carpeta, crea un archivo README.md que contenga el enlace al archivo y su código XML en un bloque de código markdown.*

2. Modificadores de acceso y Encapsulamiento en C# (Prioridad Alta)

- ***Ejercicio práctico:*** *Declara en la clase Cadete un campo privado private int _energia; y expón una propiedad pública public int Energia { get; private set; } que impida modificar la energía directamente desde fuera sin pasar por el método GastarEnergia().*

3. Polimorfismo y Métodos Abstractos / Override (Prioridad Media)

- ***Ejercicio práctico:*** *Escribe una clase abstracta Personaje con public abstract void Actuar(Cadete c); e implementa dos clases derivadas (Robot y Enemigo) que sobrescriban (override) el método con valores de daño y consumo de energía distintos.*

4. Uso de Interfaces (IInteractuable) (Prioridad Media)

- ***Ejercicio práctico:*** *Define la interfaz IInteractuable y crea dos clases (CajaDeSuministros y PuertaBloqueada) que implementen el método Interactuar(Cadete c) con efectos diferentes sobre el estado del cadete.*

5. Consultas LINQ sobre List<T> (Prioridad Media)

- ***Ejercicio práctico:*** *Dada una lista List<Objeto> inventario, escribe una consulta LINQ que ordene los objetos por poder de mayor a menor y filtre solo aquellos cuyo nombre comience con la letra 'E'.*

6. Conclusión

*Has demostrado una comprensión clara de la estructura fundamental de la **Programación Orientada a Objetos** (clases, objetos, herencia, polimorfismo, interfaces) y del manejo básico de la plataforma **GitHub** (creación de carpetas, enlaces relacionales y Markdown).*

Puntos a reforzar:

1. *Profundizar en los detalles de **encapsulamiento** y control de estado privado en C#.*
2. *Mantener clara la distinción conceptual entre **almacenar archivos binarios** y **editar texto/código** dentro de repositorios.*